

Dpto. de Electrónica y de Telecomunicaciones

Facultad de Ingeniería

Universidad de Antioquia



DESAFIO 1

Augusto Enrique Salazar Jimenez

Mateo Rodriguez Yepez

TABLA DE CONTENIDOS

1.INTRODUCCIÓN	3
2.EL TABLERO	4
2.1.CREACIÓN DEL TABLERO.....	5
2.2.USO DE MEMORIA DINAMICA.....	6
3. PIEZAS Y SU REPRESENTACIÓN.....	7
3.1 INDEXACION Y BIT MASKING.....	9
4 MOVIMIENTOS.....	11
4.1 DIBUJAR(DRAW).....	11
4.2 LIMPIAR(ERASE).....	13
5. ROTACIÓN.....	15
6. COLISIONES.....	15
7. CLEAR LINES.....	16
8.GAME OVER.....	17
9.BIBLIOGRAFÍA.....	18

INTRODUCCIÓN

En este documento se encontrará, de manera resumida, la forma y el flujo de trabajo para el desarrollo del desafío. La idea principal de mi desafío fue tratar de tener un entorno más limpio y ordenado, y es por eso que para cada header traté de hacerle su propio branch.

Aquí, en este documento, junto con las diapositivas del video, se resumen las partes más importantes, como lo son imágenes para un mayor entendimiento y fragmentos de código que son de vital importancia para la comprensión de este mismo.

En los commits se puede verificar también la edición del documento, como también la depuración del código y el seguimiento constante de este mismo.

Sin más que decir, esta es la documentación del desafío.

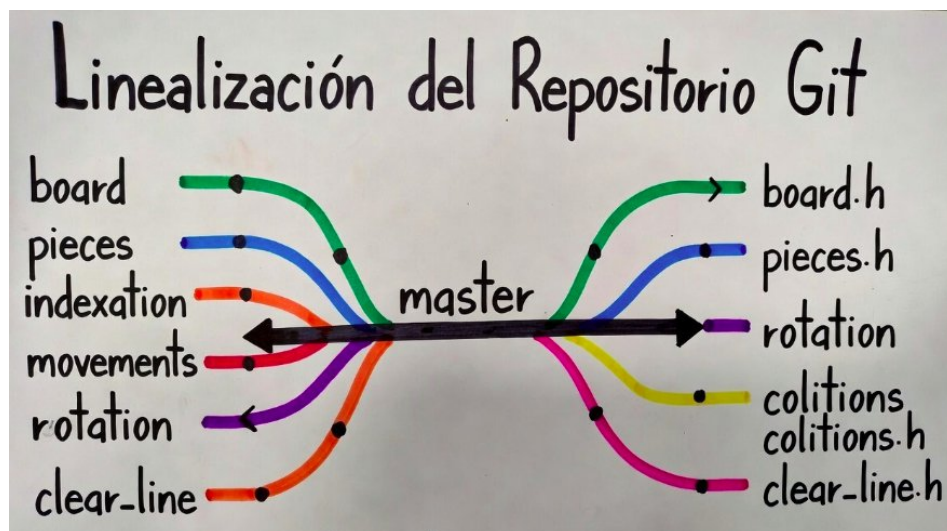


Figure 1. Representación Gráfica del flujo de trabajo branch y headers.

EL TABLERO

¿Cómo puedo hacer y representar el tablero mediante caracteres "." y "#" cumpliendo las condiciones dadas?

Realmente no trabajaremos con ese tipo de archivos; esos símbolos char únicamente serán la representación por consola del programa a desarrollar.

La verdadera esencia y lo fundamental del tablero es la representación que tendremos y el tipo de variables que utilizaremos para ello. Pero debido a una obligación de entrega, tendremos que trabajar a nivel de bits con operadores bitwise (bit a bit), así que la mejor manera de manejarlo es trabajar con enteros, más directamente con su valor binario; es decir, que lo que nos interesa es el manejo de los bits.

Nosotros, por básica informática, sabemos que un entero tiene un valor en bytes que lo define la arquitectura en la que se emplea dicho valor informático, teniendo así que el valor básico informático de un entero es 4 bytes.

4 bytes en binario serían 32 bits disponibles, viéndolo de una forma gráfica.

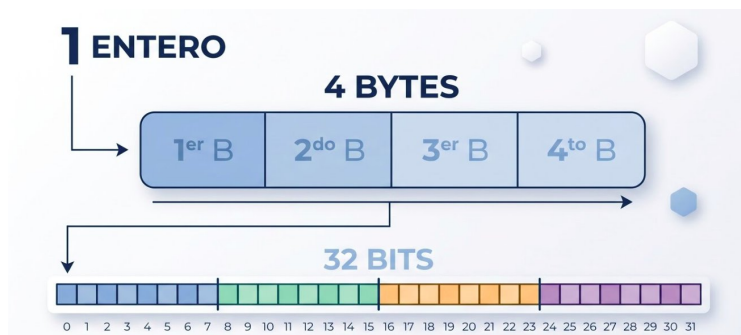


Figura 2. Representación gráfica de un entero a binarios.

Si somos lo suficientemente atentos, podemos notar que un entero a nivel de bits se parece a una fila con 32 columnas. Y si nos llevamos esa lógica, podemos decir que si tuviéramos 1 byte, tendríamos una fila con 8 columnas.

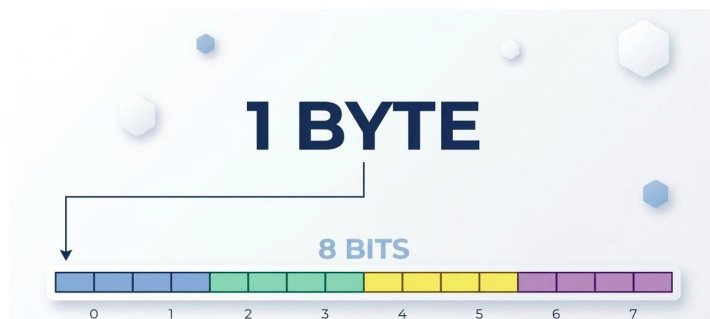


Figura 3. Representacion Gráfica de un 1byte.

Entonces, nuestra lógica nos dice que el tipo de variable debe ser de 1 byte. Y esa variable es la variable char → 1 byte.

Ya tengo las columnas mínimas (ancho) de nuestro tablero de Tetris, pero las otras filas, ¿cómo podríamos nosotros representarlas a nivel de bits?

Normalmente tenemos la costumbre de malinterpretar lo que significa una matriz a nivel de memoria y pensamos que se basa en filas y columnas, pero en realidad son posiciones de memoria que están contiguas, unas pegadas a las otras, y mediante esa contiguidad podemos crear matrices gráficamente.



Figure 4. Representacion en memoria de una matriz.

Siguiendo esa lógica, podemos decir que podemos crear un tablero con un arreglo dinámico de chars, que primordialmente los trabajaremos a nivel de bits y luego los convertiremos a caracteres en consola.

Un punto importante a tener en cuenta es que el ancho de nuestro tablero será múltiplo de byte (8 bits). De una forma matemática, nuestro ancho estará dado por:

$$N * \text{Byte} = N * (8) \text{ bits}$$

CREACIÓN DEL TABLERO.

En la lógica primordial, tenemos que un usuario ingresará por consola un ancho y alto mínimo, y nuestro programa deberá convertir eso a un tablero de bits, no sin antes pasar por nuestras transformaciones.

$$\text{Bytes_por_filas} = (\text{ancho}) / (8) \text{ \#Bytes}$$

Con el resultado de nuestra primera transformación, podemos asignar de cuántos bytes será nuestro entero.

```
Unsigned char tablero[alto][bytes_por_filas]
```

La transformación del alto me dará la dimensión de nuestro array y, por ende, nuestro tablero de Tetris (convirtiéndolo mentalmente a un tablero).

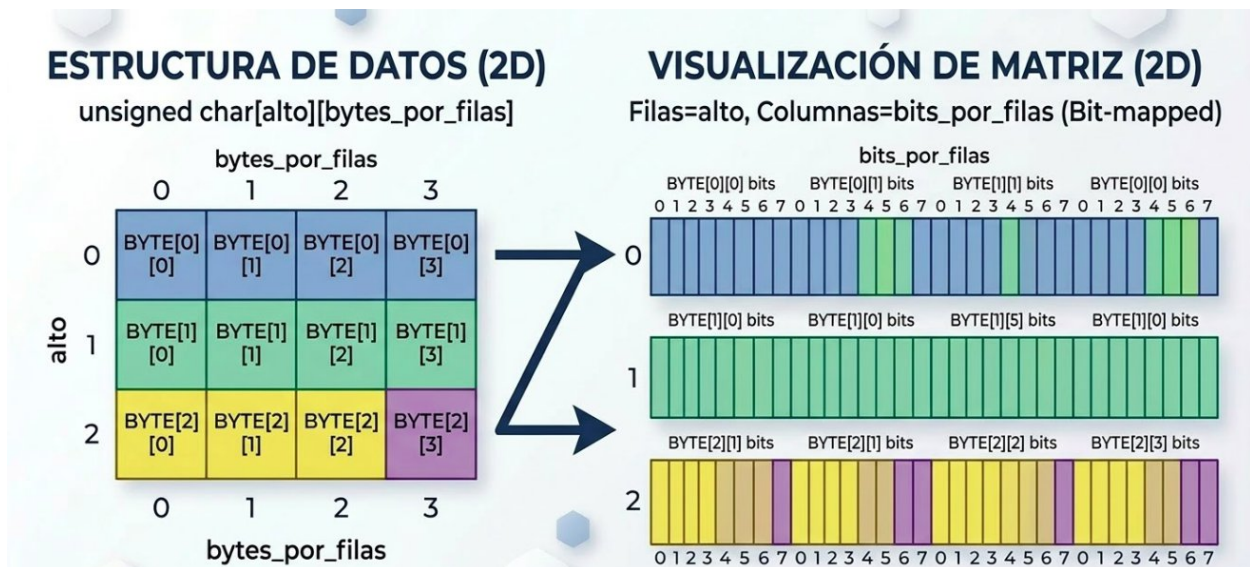


Figure 5. Representación Gráfica de una matriz a una matriz de bits

USO DE MEMORIA DINAMICA

Necesitamos asignar memoria dinámica para crear nuestro tablero. Esto lo hacemos de la siguiente forma:

Primero, necesito un puntero que me apunte a un puntero, que a su vez apunte a filas, y ese puntero a filas que me apunte a las columnas:

```
Unsigned char ** tablero;
```

Después, reservo un arreglo de punteros que me apuntarán a cada fila:

```
tablero = new unsigned char*(alto)
```

Luego, reservo los bis que utilizaré para cada fila

```
tablero[i] = new unsigned char[bytes_por_filas]
```

Algo gráficamente como:

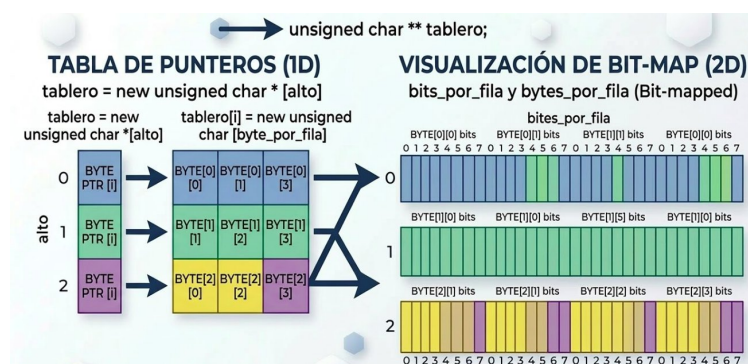


Figure 6. Representación Gráfica de punteros sucesivos

Al final unicamente rellenamos los valores con 0 para que nos quede algo tipo:

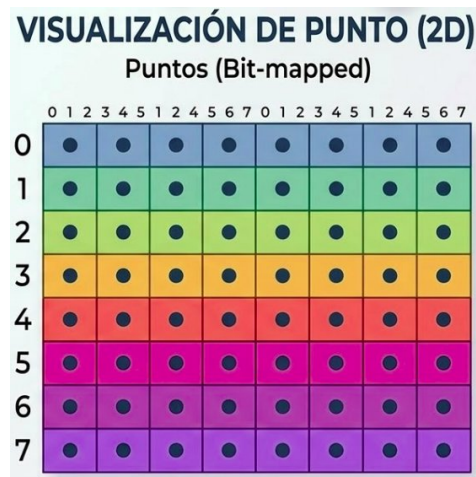


Figure 7. Creacion Tablero tetris con 8 de ancho y 8 de alto llenado con 0's

PIEZAS Y SU REPRESENTACIÓN

Ahora nos enfrentamos a la representación y lógica de nuestros tetrominós (7 en total).

Primero, debemos definir cómo los representaremos a nivel de bits, ya que son estos los que nos darán una mecánica fundamental en nuestro desarrollo: el bit masking. Así que lo primordial son nuestros tetrominós y su representación en binario.

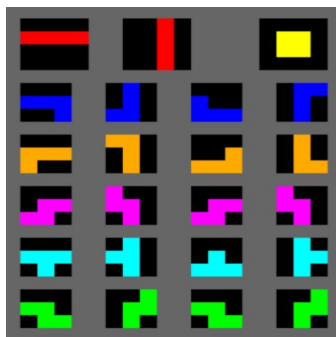


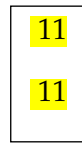
Figure 8. Tetriminos y sus rotaciones.

Utilizando su representación de bits, podemos decir que en las matrices, si hay color, son 1's, y si hay negro, son 0's, teniendo así una representación acertada en bits.

– I (Línea o Pieza I): Recta de cuatro bloques (1x4).

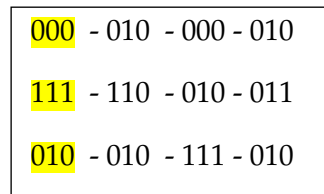
0000	- 0010
1111	- 0010
0000	- 0010
0000	- 0010

O (Cuadrado): Bloque de 2x2.



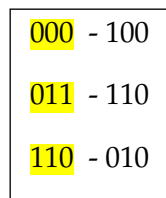
—

T (Pieza T): Forma de T. Versátil para encajar en espacios irregulares (3x2).



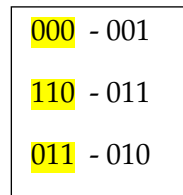
—

S (Pieza S): Forma de S, se orienta hacia la derecha (3x2).



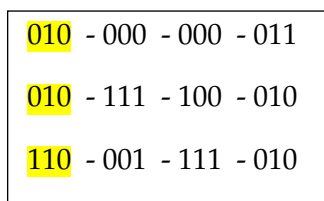
—

Z (Pieza Z): Forma de Z, se orienta hacia la izquierda (3x2).



—

J (Pieza J): Forma de L invertida o con gancho a la izquierda (2x3).



L (Pieza L): Forma de L estándar o con gancho a la derecha(2x3).

010	- 000 - 110 - 000
010	- 111 - 010 - 001
011	- 100 - 010 - 111

En los ejemplos anteriores vemos su representación binaria y, aparte, las rotaciones que ciertos tetrominós tienen. Ahora sigamos con la siguiente cuestión:

Ya tenemos nuestro tablero, pero ¿cómo añado mis piezas a mi tablero?

Es aquí donde utilizamos otra mecánica del programa: el bit masking. Pero antes de utilizar el bit masking, la verdadera pregunta es: ¿cómo me indexo para que el bit masking sea ejecutado correctamente? Y es aquí donde nos enfrentamos a otro gran problema.

INDEXACION Y BIT MASKING.

Como opción principal, toda pieza ingresará en la mitad de nuestro tablero y de ahí podemos hacer la indexación.

Sabiendo que nuestro tablero tiene tamaño:

Tablero[alto][ancho_bits]

Tenemos que la mitad de nuestro tablero será

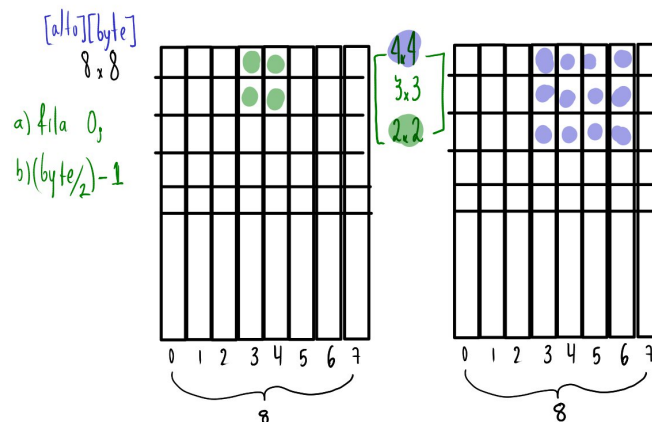


Figure 9. Esquema propio de indexación de tetriminos

Utilizo que mis figuras tengan dimensiones fijas y cuadradas para mejorar las mecánicas en el bit masking, por ejemplo: 2x2, 3x3, 4x4. Y ya con esto estoy indexado.

Debido a problemas en el código, todas las fichas deben ocupar un espacio de matriz de 4x4 aunque algunas únicamente ocupen un espacio de 2x2.

Pero para mejorar considerablemente mi indexación puedo utilizar variables de posición como por ejemplo:

$$X = ((\text{byte})/2)-1; y=0$$

Ahora, únicamente me toca utilizar la mecánica de bit masking, pero no sin antes hacer lo siguiente: tengo que apuntar bien, debido a que si únicamente utilizo bit masking, me rellenará la columna de manera lineal.

Nuestra mecánica para indexarnos es la siguiente:

Para cada fila de la pieza, apuntar a la fila del tablero.

Para cada columna de la pieza, utilizar el bit masking.

Esto se hace utilizando únicamente ciclos e indexadores.

Aquí tenemos que hacer algo sumamente importante: el bit masking. Pero ¿qué es lo que realmente se debe hacer?

Primero, tenemos nuestro tablero. Nuestro tablero está lleno de 0's, y nuestras fichas son matrices que tienen bits activos (1's) y bits inactivos (0's). Así que lo que tenemos que hacer es recorrer y comparar esos bits utilizando el operador "|" (OR), el cual nos dice que si uno de los bits está activo, entonces el resultado será un bit activo.

Pero lo primordial es que tenemos que movernos dentro de las columnas, y eso lo hacemos con los operadores de desplazamiento (shift):

"<<" (desplazamiento a la izquierda)

">>" (desplazamiento a la derecha)

Cuando nos movemos y comparamos, estamos indexando nuestra ficha completa.

1 << 0 → 00000001

1 << 1 → 00000010

1 << 2 → 00000100

1 << 3 → 00001000

1 << 4 → 00010000

LA FUNCION Y EL BRANCH INDEXATION QUEDAN INUTILIZADOS DEBIDO A PROBLEMAS QUE TENIA CON ESA BRANCH Y HEADER., PORQUE LITERALMENTE ESTABA HACIENDO UN TRABAJO INNECESARIO.

MOVIMIENTOS

Teniendo mi tablero y mi pieza listos, lo siguiente que puedo hacer es empezar a buscar una solución para mover mis fichas. Primordialmente, puedo pensar en la indexación: si me indexo con coordenadas variables, se puede tener mayor control y facilidad.

Antes de mover, claramente debo primero dibujar mi matriz y también eliminar mi matriz dibujada. Así que, optimizando un poco mi código, no utilizo mi header de indexación debido a que necesito que mis coordenadas sean variables, porque después de cada movimiento las coordenadas cambian y no se pueden resetear siempre a un mismo punto.

Llevando esta idea en mano, puedo decir que tengo estos flujos de trabajo:

elimino tetrimino anterior

Movimiento de tetrimino

dibujo tetrimino actual

Lo primero que tengo que hacer a la hora de eliminar o dibujar un tetrimino es indexarme, y eso lo puedo hacer recorriendo el índice de mi tetrimino $[i][j]$. En este caso, ambos son de 4×4 .

Lo siguiente es verificar qué fichas están activas o inactivas.

Si la ficha está activa, entonces inicio mis coordenadas variables y empiezo a moverme entre filas y columnas con ayudas auxiliares de las variables i, j .

Después de indexarme de esa manera, empiezo a buscar cuántos bytes y bits tienen las columnas con operaciones sencillas, para poder hacer los bitwise después.

Cuando empiezo a recorrer los bits, hago un bitwise que utiliza la siguiente lógica:

$A | B$

nos dice que si un bit está activo, entonces todos estarán activos.

DIBUJAR(DRAWN)

Dibujar, en este caso, significa que a nuestro tablero original de Tetris le pondremos el respectivo tetrominó con su bitwise para que se edite el tablero en cada movimiento.

Al inicio, tenemos que nuestro tetrominó está en coordenadas [i][j]. Pero necesitamos pasarlo a coordenadas del tablero (x, y):

```
int fila = y + i;  
int column = x + j;
```

Un ejemplo rápido de esta practica es si, por ejemplo, nuestra ficha esta en la posicion

```
x = 10 ; y = 5  
Y estamos en el bloque de la pieza  
i = 2 y j = 1
```

Entonces lo que decimos es que el bloque del tetrimino debiera de ir en el tablero en la posicion

```
Tablero[12][6]
```

De aquí le damos la bienvenida a cómo debo enmascarar esa indexación que hice, y es por eso que debemos recordar cómo está organizado mi tablero.

Recordemos que para cada:

```
Unsigned char tablero[alto][bytes]
```

Cada unsigned char tiene 8 bits

Es decir que si tuvieramos un tablero con 16 de ancho serian

```
2 bytes = 16 bits
```

Que visualmente se verian asi

```
byte0      byte1  
00000000 : 00000000  
8 columnas 8 columnas
```

Sabiendo como se organiza mi tablero entonces debemos saber en que byte(columna entre byte) está ubicado mi tetrimino y eso lo hacemos con

```
int byte = column / 8;
```

Despues que sabemos en que byte está debemos saber en que bits especifico está y eso lo hacemos con una division modular muy facil que es

```
int bit = column % 8;
```

Ahora seguimos con la forma en cómo trabajaremos nuestro bitwise.

La idea principal de nosotros es encender bits, ya que en las columnas de bits todos están apagados. Entonces, lo que debemos hacer es eso, pero sin interrumpir los demás, es decir, únicamente hacerlo en nuestro espacio del tetrominó.

Sabemos que, por ejemplo, una fila tiene la siguiente forma:

```
00000000
```

Y queremos activar el bit numero 4, lo que haremos será lo siguiente

Tomamos un numero con un bit activo ejemplo 1

```
1 = 00000001
```

Y lo que hacemos es pasar ese a la posicion 4 mediante shift asi

```
1 << (4-3)
```

Y lo que me hara será

```
00001000
```

Entonces lo unico que hago es decir que

```
0 | 1 << (4-3)
```

Nota: el | es el “|” la fuente lo cambia

Y me dara

```
00001000
```

Siguiendo esa lógica podemos decir que para activar un bit en una columna y fila especifica(x,y) utilizamos a:

```
tablero[fila][byte] |= (1 << (7-bit))
```

LIMPIAR (ERASE)

Ahora seguimos con lo inverso a dibujar en tablero, y es eliminar lo que hay en un tablero.

De manera lógica, haremos lo contrario hasta el punto de desactivar la columna activa, y lo que haremos será apagarla. Eso lo hacemos de la siguiente manera:

Imaginémonos que tenemos este bit:

10000100

Lo que queremos hacer es que el bit activo en el 3 se apage y para eso lo que haremos es lo siguiente:

Ubicamos el bit en la posición querida en este caso

1=00000001

1<< (2)

00000100

Lo que hacemos despues es invertir todo con ~

~(1<< (2))

11111011

Utilizamos el operador & que nos dice que un bit estará activo unicamente si ambos lo estan entonces

10000100

&

11111011

=

100000100

Siguiendo esa logica utilizamos entonces:

tablero[filas][byte] &= ~(1 << (7-bit))

Al final, teniendo la lógica de dibujar y eliminar piezas en el tablero, podemos simplemente mover nuestras fichas mediante coordenadas X y Y. Como por ejemplo:

Para movernos a la derecha, podemos utilizar X++

Para movernos a la izquierda, X—

Y para bajar, únicamente Y++

ROTACIÓN DE FICHAS.

Lo que haremos para rotar nuestra ficha será lo siguiente: nosotros tenemos fichas predefinidas en el header piezas, entonces lo que haremos será llamar de manera aleatoria una de ellas y trabajaremos con ellas, creando una copia para que luego la rotación hecha por la copia sea actualizada a la original.

Lo primero que haremos será llamar a nuestra ficha predefinida y crearemos una copia, la cual rotaremos. La rotaremos gráficamente como se ve en la imagen, así de 90 grados.



Figure 10. Rotación de matriz 90°.

La idea principal de nuestra rotación es rotación por índices, donde para cada posición (0,0) se me traslade a (0,3), y para cada posición (3,0) se me traslade a (0,0). Podemos ver que tanto para índices [i][j] se mueven así que

Para cada posición [i][j] se mueva [j][3-i] posiciones

COLISIONES

Más que una función, las colisiones son una comprobación que hacemos para saber si hay algo debajo de mí o no, y así poder hacer dos cosas: bajar con Y++ o simplemente crear otra pieza aleatoria para mi Tetris.

Es por eso que para este header utilizaremos el tipo char como booleano, porque lo que nos retornará es si podemos bajar o no lo podemos hacer.

Nuestro flujo para verificar si podemos bajar será el siguiente:

Recorremos con coordenadas nuestros bits y únicamente nos interesan los bits que están activos.

Luego nos indexamos en nuestra columna de bits con

```
X + j.
```

Y luego "miramos" lo que hay debajo de nosotros con nuestras coordenadas

```
Y + i + 1.
```

Comprobamos que nuestra colisión no pase del suelo, sabiendo que nuestra "mirada" hacia abajo sea menor que el alto de nuestro tablero.

Luego verificamos si lo que hay debajo está activo o inactivo, y eso lo hacemos con el mismo bitwise del draw y el erase.

Utilizando la lógica de que si el bit está activado, entonces me saque un bool que me diga que no se puede bajar. Si no hay bits activos ahí, entonces todo sale bien con un return true;

CLEAR LINES

Esta es una de las partes del código importantes ya que tiene el importante flujo de trabajo que es el siguiente:

Buscar filas completas

Eliminar las filas completas

Bajar todo lo que está arriba

Crear una fila vacía arriba

Eso lo haremos de la siguiente manera:

Recorremos todas las filas con operaciones básicas de indexación

Luego suponemos que nuestra fila está llena, lo que hacemos es verificar si nuestra fila si esta llena y eso lo hacemos con:

```
if(tablero[i][j] != 255)
```

Utilizamos el 255 por que es el valor binario completo de una columna de 8 bits en 1, es decir,

```
255= 11111111
```

Si eso no se cumple entonces decimos que nuestra fila no está llena

Luego de que verificamos que la fila este llena lo que hacemos es mediante otra variable auxiliar recorrer filas inferiores con

```
for(int k = i; k > 0; k--)
```


Eso lo que hace es que recorramos las filas de la manera

```
fila i    ← fila i-1  
fila i-1  ← fila i-2  
fila i-2  ← fila i-3
```

Luego copiamos los byte a byte (filas a filas)

```
tablero[k][j] = tablero[k-1][j];
```

Y por ultimo limpiamos la primera fila con:

```
tablero[0][j] = 0;
```

GAME OVER

Para terminar el juego lo unico que haremos será una verificación utilizando la lógica de que será true si la pieza cabe, es decir, puede spawnear otra pieza

Y false si la pieza ya no cabe.

Lo que haremos primero será recorrer de manera indexada mi tetrimino , revisamos unicamente los bits que están activos, es decir, donde va mi pieza , luego debemos ubicarnos en nuestra tablero y eso lo hacemos con

```
int tablero_x = x + j;  
int tablero_y = y + i;
```

Luego ubicamos nuestros bytes (filas) y nuestras bits (columnas)

```
int byte = tablero_x / 8;  
int bit  = tablero_x % 8;
```

Luego verificamos si en esa posicion hay bits activos mediante comprobación con bitwise:

```
if (tablero[tablero_y][byte] & (1 << (7-bit)))
```

Si la condición sale verdadera nuestro juego culminara con un

GAME OVER

BIBLIOGRAFÍA

Aquí pondre todo el material que encuentre en internet el cual me ayudo para entender y culminar el desafio

<https://stackoverflow.com/questions/18591924/how-can-i-use-a-bitmask>_(web bitmasking)

<https://www.luisllamas.es/cpp-operadores-bitwise/> (web bitwise)

https://youtu.be/ffPOA7UUDAs?si=hXY74N-_bURaAW1u (video de bitmasking)

<https://youtu.be/PoajUl17KSw?si=Z9Sh-T2PTnBOGT9c> (video rotar matrices)